Smart Nursery Guardian: Project Plan and Technical Documentation

Mind Cloud Summer Training - Team 1

September 12, 2026

1 Two-Week Project Execution Plan

The project lifecycle is structured into two systematic phases to ensure parallel progress across hardware prototyping, software design, and PCB layout.

Project Plan				
Technical Tasks	Specialization	Member	status	Ended by Date
MCU(Tinkercad Simulation)	HW	Dima	✅ Completed	4/9/2026
MCU(Proteus Simulation -Bonus)	HW	Dima	Canceled	
Breadboard Prototyping	HW	Dima	✅ Completed	4/9/2026
PCB Design	HW	Dima	✅ Completed	12/9/2026
Microcontroller Program	SW/HW	Jana	✅ Completed	4/9/2026
Serial Communication	SW/HW	Rana	✅ Completed	9/9/2026
Data Preparation(Cleaning Audio-Augmentation-Feature Extraction)	SW	Sarah	✅ Completed	4/9/2026
GUI	SW	Rana	✅ Completed	3/9/2026
ML Model Training(Training-Testing and Confusion Matrix-Saving File)	SW	Dhuha	✅ Completed	5/9/2026
Telegram	SW	Rana	✅ Completed	6/9/2026
Integrating Work				
Machine Learining : Processed data & Training Model	SW	Dhuha/Sarah	✅ Completed	12/9/2026
ML model with GUI & Telegram	SW	Rana/Dhuha/Sarah	✅ Completed	12/9/2026
Prototype Breadboard with MC Code	HW/SW	Jana/Dima	✅ Completed	12/9/2026
ML Serial with MC Code	SW/HW	Rana/Jana	✅ Completed	12/9/2026
Non-Technical Tasks	Specialization	Member		
Progress Reports	Any	Any	✅ Completed	3,6,9 september
Cost Management	HW	Dima	✅ Completed	31/8
Video Preparation	Any	Any	✅ Completed	Time to start 11/9 0:00

Project Plan

2 System Architecture and Technical Approach

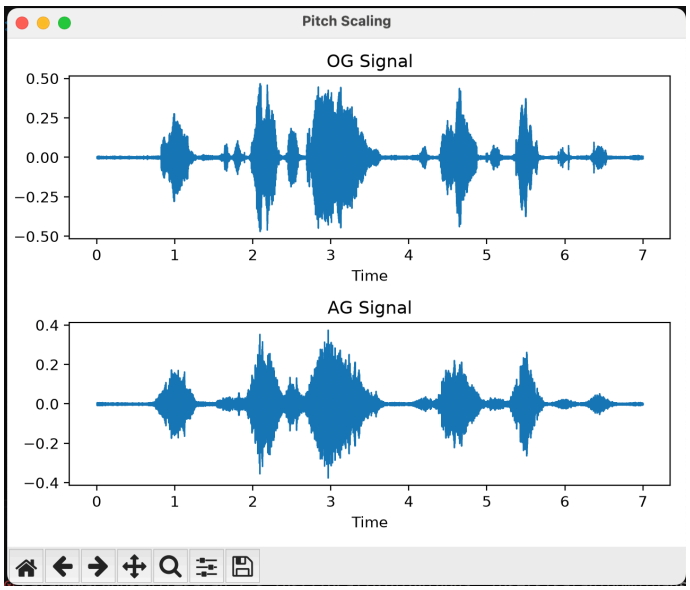
The system relies on a hybrid architecture dividing tasks based on computational capability:

Laptop (Python):

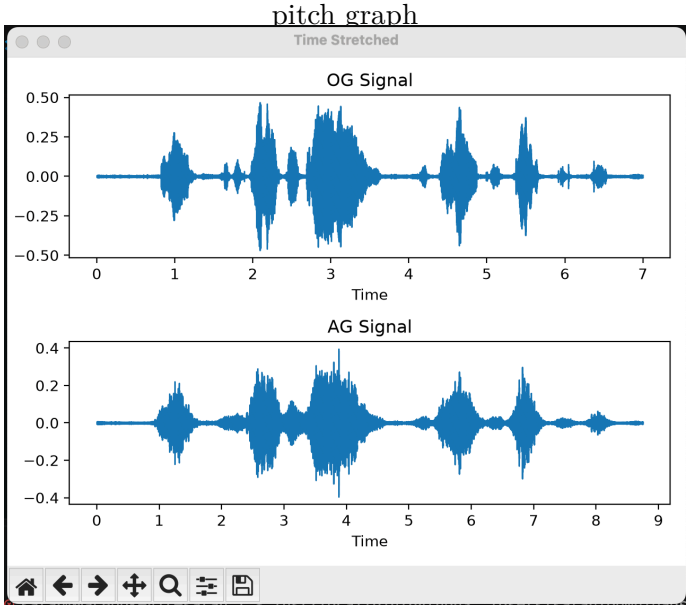
Handles heavy audio processing, machine learning inference, GUI rendering, and Telegram cloud alerts.

Machine Learning Pipeline:

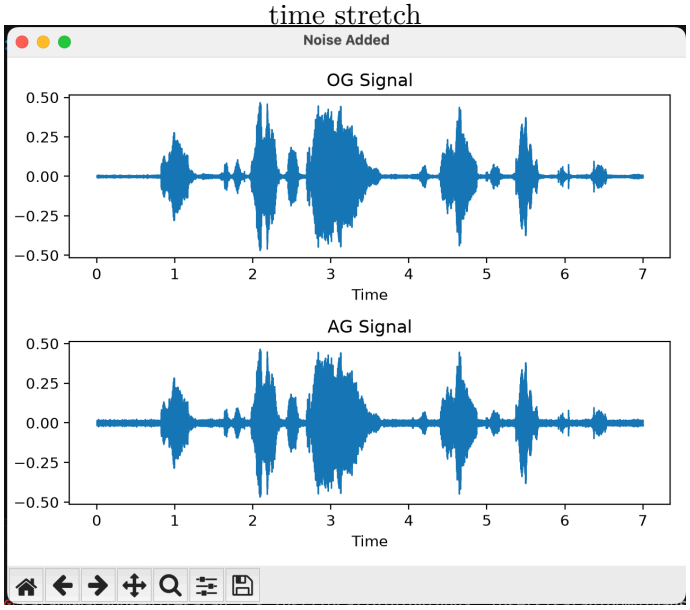
- **Audio Processing Setups:**Audio Processing Setup: Explored key audio processing libraries like librosa, scipy.signal, soundfile, and noisereduce to manipulate audio parameters and construct a comprehensive preprocessing pipeline.
- **Augmentation and Split Strategy:** Augmentation & Split Strategy: Split raw data (80% train / 20% test) before augmentation to prevent data leakage. Skipped the majority class (hungry) during augmentation to mitigate class imbalance, while expanding minority classes (discomfort, tired) using 10 techniques with 2 extreme-value boundaries per method.
- **Feature Extraction Logic:** Feature Extraction Logic: The extract_cry_features function removes background noise using spectral gating, trims silence/gasps via Voice Activity Detection (VAD) using librosa.effects.split, and returns None if no signal remains. Valid signals yield a 29-element feature vector (13 MFCC means/stds, F 0 mean/std, RMS intensity mean/std, and duration) saved as .npy files for model training. Pipeline Validation & Live Inference: Validated output quality by playing raw vs. augmented samples (sounddevice) and generating waveform comparison plots (plot_two_signals). Designed get_live_features and classify_live_record for real-time GUI integration (Rana's code), filtering out silent audio (<0.01 amplitude) before passing live features to the saved pipeline. Severe Class Imbalance: The dataset was heavily biased toward hungry. Solution: Excluded hungry from augmentation while generating balanced synthetic samples for minority classes using 10 varied transformations. Audio Artifacts & Noise Ingestion: Silent recordings, gasps, and background noise corrupted feature metrics. Solution: Implemented noisereduce paired with VAD segmentation, automatically returning None for silent inputs.
- **Training Stage:** The training data is loaded and SMOTE is applied to handle class imbalance, followed by feature selection using a preliminary Random Forest model. A machine learning pipeline is then constructed and optimized using RandomizedSearchCV over a hyperparameter grid to find the best Random Forest classifier settings.
- **Saving Trained Model:** Upon completing the grid search and identifying the best estimator, the fully trained model is serialized and saved onto the hard disk as a Pickle file (.pkl). This enables the retrieval and reuse of the trained model for subsequent testing or live deployment without retraining.
- **Testing and Assessment Stage:** The saved model is loaded and evaluated on testing data to generate predictions, followed by performance assessment via Confusion Matrices and multilabel confusion matrices. Training and testing accuracy scores are calculated alongside Macro F1 and Weighted F1 metrics to check for overfitting and bias.
- **Extracting Prediction as Outputs:** A dedicated live classification function loads the saved model to process incoming audio feature inputs and generate model predictions. The numerical prediction is then mapped via the label map dictionary into a clear textual output representing the infant's state.
- **Preprocessing, Augmentation and Training Model Results:**
  - **Optimization & CV Results:** Randomized search yielded optimal parameters ($n\_estimators = 400$, $max\_depth = 14$, $min\_samples\_split = 3$, $min\_samples\_leaf = 4$, and $max\_features = 'sqrt'$) with a cross-validation score of 0.934.
  - **Performance Metrics:** The model achieved 99.0% training and 81.8% testing accuracy (gap: 0.17), with a high weighted F1 of 0.82 contrasting a macro F1 of 0.51.



pitch graph



time stretch



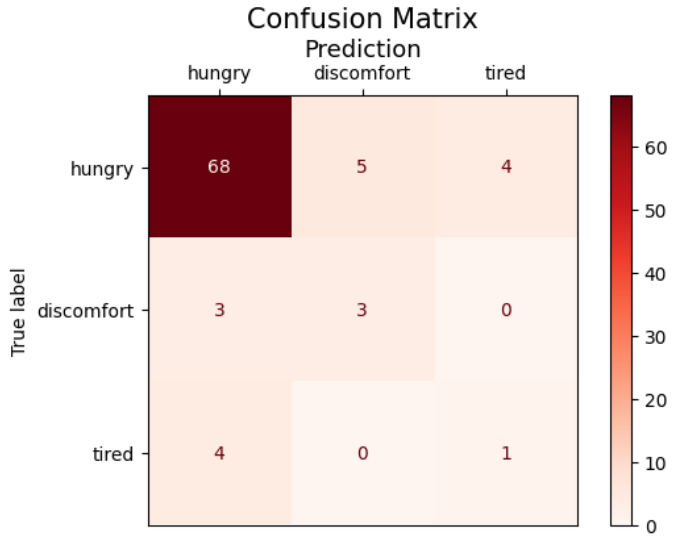
white noise graph

```
20      }
21
Problems 1   Output   Debug Console   Terminal   Ports

sarah@MacBook-Air-sarah ~ % cd "/Users/sarah/" && g++ g2.cpp -o g2 && "/Users/sarah
t viable: no overload of 'endl' matching 'double &' for 1st argument
  253 |   basic_istream& operator>>(double& __f);
      |                  ^          ~~~~~~~~~~
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/c++/v1/istream:254:
t viable: no overload of 'endl' matching 'long double &' for 1st argument
  254 |   basic_istream& operator>>(long double& __f);
      |                  ^          ~~~~~~~~~~~~~~~
/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include/c++/v1/istream:255:
t viable: no overload of 'endl' matching 'void *&' for 1st argument
  255 |   basic_istream& operator>>(void*& __p);
      |                  ^          ~~~~~~~~~~
1 error generated.
sarah@MacBook-Air-sarah ~ %
```

```
sarah@MacBook-Air-sarah Downloads % /usr/local/bin/python3 "/Users/sarah/Download
hungry Train: 305  Test: 77
discomfort Train: 21  Test: 6
tired Train: 19  Test: 5
```

splitting results



Confusion Matrix on Test Set

```
Multilabel confusion matrix is like :
 [['TN' 'FP']
 ['FN' 'TP']]
hungry Confusion Matrix :
 [[ 4  7]
 [ 9 68]]
discomfort Confusion Matrix :
 [[77  5]
 [ 3  3]]
tired Confusion Matrix :
 [[79  4]
 [ 4  1]]
Training Accuracy Score : 99.0%
Testing Accuracy Score : 81.8%
Accuracy Gap : 0.17
Macro Score for testing (Indication of model's overall performance)  :0.51
Weighted Score for testing (Idication of model's performance) : 0.82

Good Fit: balanced performance and small gap
No Bias
```

Model Testing Results and Scores

GUI Rendering:

- **GUI Foundation:** Started with GUI by laying the base skeleton for incorporating the serial and ML model into later, and added more features along the way.
- **Audio Input Idea:** First it was an idea to receive audio as an input internally without actually showing it on the GUI, but then it was decided that a simple GUI for sound that resembles a voice memo like that of mobile recording would benefit testing purposes and moms who want to make sure that the audio input is working. This was built using a live bar graph (matplotlib bars embedded in the Tkinter window via FigureCanvasTkAgg), fed by a PyAudio stream reading 1024-sample chunks and updating the amplitude levels continuously.
- **Sensor Labels:** Added the labels for the features that were supposed to receive their inputs from the hardware, such as fan, light, awake, gas, and temp values. Their values were randomized at first for testing purposes, and later set to a certain value to test triggering alerts and Telegram notifications. The baby state label was set and randomized as well.
- **Clock:** Then for aesthetic purposes and for practicality as well, a timer was added in the GUI to see the clock, updating every second through the '.after' method.
- **Serial Connection:** Then, after the microcontroller code was done, it was time for the serial connection, where the library pyserial was used. It was at first for just receiving the line for the readings, decoding it, then string manipulation to extract the data we need (Temp, Gas, Awake, Fan, Light, parsed from the raw line). Added a few lines to the microcontroller code for printing that line repeatedly, and suggested removing all loops and delays in order not to lag the serial data input through Python, and updating the data regularly through GUI.
- **Telegram Bot:** Then the Telegram bot was set using token and using JS protocol (JSON) requests and responses, triggered through 'trigger_alert' when the gas value crossed the threshold.
- **Incorporating the ML Model:** After the ML code was done, it was time to incorporate it to the GUI. I faced an issue, which was that the audio input wasn't really suitable for detecting the cries. When I first added the data input part, I reached a threading and queueing approach, then decided to instead replace it with a simpler approach, which was using an '.after' method. Instead of continuously inputting audio, input continuous little chunks of data input that is good enough for a simple voice memo logic, which didn't quite align with detecting the cries at all. So I searched and found that I could add a rolling audio feature by adding a deque, holding a 6 second window (96000 samples at 16kHz) that slides forward by 1 second once full, and gets passed through 'extract_cry_features' (noise reduction, VAD, MFCCs, pitch, RMS intensity, duration) before being classified by the saved model.
- **Feature Additions Tied to ML Input:** I then added servo stop/start feature in case of crying, running on every 64ms chunk through 'update_servo' so the response is instant, separate from the 6 second window used for ML classification. Also added buzzer trigger when tired, and other features that depended on the ML input.
- **Window Management Bug:** After bringing the whole project together, I found that closing the unwanted windows after the baby state changed wasn't handled, as when the baby was hungry, a video window was triggered on top of the main GUI, but if the baby state changed to tired before the video ended, the video resumed without interruption, which was wrong. So that was handled, the video window now gets destroyed and reset to None whenever the state moves to discomfort, tired, or fine.
- **Priority Handling:** Priorities, as well as the gas alert, dominated over the others, the gas alert window takes over and closes the video window if gas exceeds the threshold, even mid hungry state.
- **Suggestion Label & Dismiss Button:** Then I added a suggestion label in the end, updating every 500ms based on the current baby state, and a dismiss button, if the parent actually read the baby state label and found the baby to be, say, hungry, and just fed the baby, they could just dismiss the hungry so the baby state will again be fine.

- **Known Issue:** The '.after' method, used throughout the project (waveform updates, state checks, suggestion updates, serial polling, all running on their own recursive '.after' loops), was also updating the GUI continuously, which caused a bit of lag.

Microcontroller (Firmware):

Executes low-level, real-time sensor polling (motion, temperature, gas, light) and direct hardware actuation (fan, buzzer, servo motor).

- **Firmware Scope:** The firmware handled the local sensors and actuators while the laptop side handled cry detection and classification.
- **Testing Environment:** The cry and tired flags were treated as external inputs from the higher-level system, but the serial parsing that sets them was not part of this firmware work. The code was first tested in Tinkercad simulation and later tested on the physical hardware.
- **Pin Configuration:** The firmware configured the pins for the PIR sensor, LDR light sensor, gas sensor, temperature sensor, buzzer, RGB LED, room LED, motor driver, and servo. It treated the PIR sensor as a digital input, the LDR, gas, and temperature sensors as analog inputs, the buzzer and LEDs as digital outputs, the motor enable pin as a PWM output, and the servo as a servo-library output.
- **Gas Safety:** For gas and smoke safety, the gas sensor was read with analogRead and compared with a threshold. If gas exceeded the threshold, or if the tired flag was active, the buzzer was turned on. An else branch was added to turn the buzzer off again when the condition was no longer true.
- **Temperature Control:** For temperature control, the raw analog value was converted to voltage and then to Celsius. The RGB LED and cooling fan were controlled together: below 25 degrees the blue LED was on and the fan was off, from 25 to 30 degrees the green LED was on and the fan ran at half speed, and above 30 degrees the red LED was on and the fan ran at full speed. Motor direction was set with the IN1 and IN2 pins, and fan speed was set with analogWrite on the enable pin.
- **Motion Detection:** For motion detection, the PIR sensor was sampled every second inside an 8-second window. If four or more motions were detected in that window, the baby was considered awake. The timing used millis instead of delay so the rest of the firmware could keep running.
- **Room Lighting:** For room lighting, the LDR value was read with analogRead. The room LED turned on only when the room was dark and either the baby was awake or a cry was currently detected. An else branch turned the LED off again.
- **Crib Rocking:** For servo rocking, when cry was active the servo cycled through 0, 90, 180, and 90 degrees, holding each position for about one second. The timing used millis instead of delay, and the timing variable was reset when crying stopped.
- **Challenges and Solutions:** The main challenges were finding suitable thresholds for normal conditions, making sure the buzzer and LED turned off again after being triggered, deciding what should be analog and what should be digital, converting the temperature sensor reading to Celsius, and avoiding blocking loops and delay calls. Blocking code caused serial commands to be missed because the main loop could not run continuously. This was solved by replacing loops and delays with if conditions that run once per main loop, keeping state variables outside the loop, and using millis for timing. Another challenge was the real hardware LDR sensor, which read low values as light and high values as darkness, so the light condition was changed from less than the threshold to greater than the threshold compared with simulation.
- **Code Implementation Details:** Important code details include the use of millis and unsigned long for timing, explicit else branches for buzzer and LED off states, analogRead for LDR, gas, and temperature sensors, digitalRead for PIR, analogWrite for fan speed, the Servo library for crib rocking, the 8-second PIR window with 1-second sampling, the temperature-to-fan and LED mapping, the light logic based on darkness plus awake or cry, and servo cycling only while cry was active.

- **Final Implementation:** The firmware was successfully implemented in Tinkercad simulation and gave accurate results for all test cases for all sensors and conditions. After that, I helped test the physical hardware and debug its connection with the microcontroller code. I also helped with some component libraries in Altium and contributed to hardware testing and debugging.

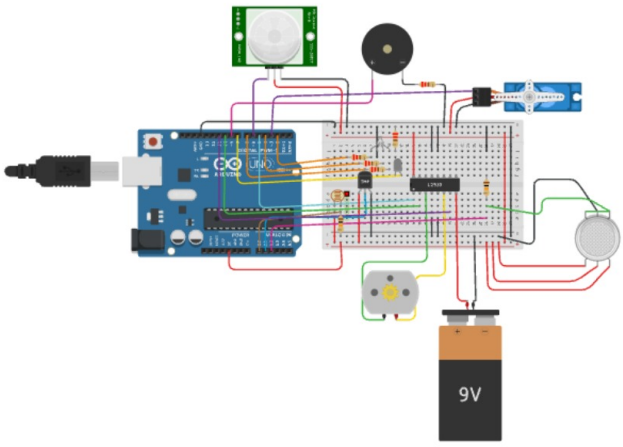
Hardware:

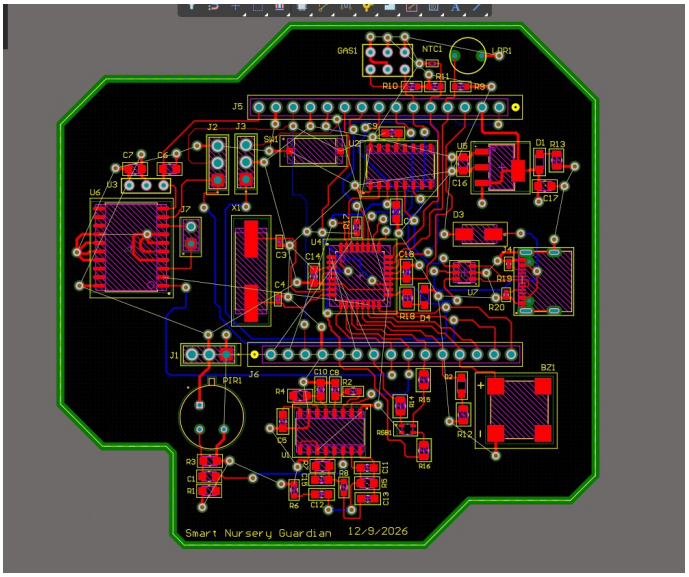
- **Prototype Breadboard and Simulation:** Prototype Simulation and Physical Assembly The hardware development phase began with virtual prototyping to validate circuit logic before physical assembly. Initially, the system architecture was designed around an STM32 Black Pill using Proteus; however, persistent software crashes and simulation instability necessitated a pivot to an Arduino Nano. To ensure a reliable testing environment, the complete circuit was successfully modeled and verified in Tinkercad, utilizing the architecturally similar Arduino Uno as a functional stand-in for the simulation. Following this virtual verification, the physical prototype was constructed on a breadboard strictly according to the Tinkercad schematic. The transition from simulation to real-world hardware was seamless, with the physical assembly operating perfectly on the first iteration, thereby establishing a fully verified foundation for the subsequent custom PCB layout and routing phase.

- **Custom PCB Design and Layout:** The development of the custom Printed Circuit Board (PCB) required extensive research to transition from pre-assembled modules to discrete onboard components. Significant effort was dedicated to identifying specific manufacturer part numbers (MPNs) through LCSC, acquiring accurate component libraries, and defining the precise electrical connections for the Arduino Nano's core architecture—including the ATmega microcontroller, crystal oscillator, USB-C interface, and reset circuitry—as well as the discrete elements of the voltage regulator and PIR sensor.

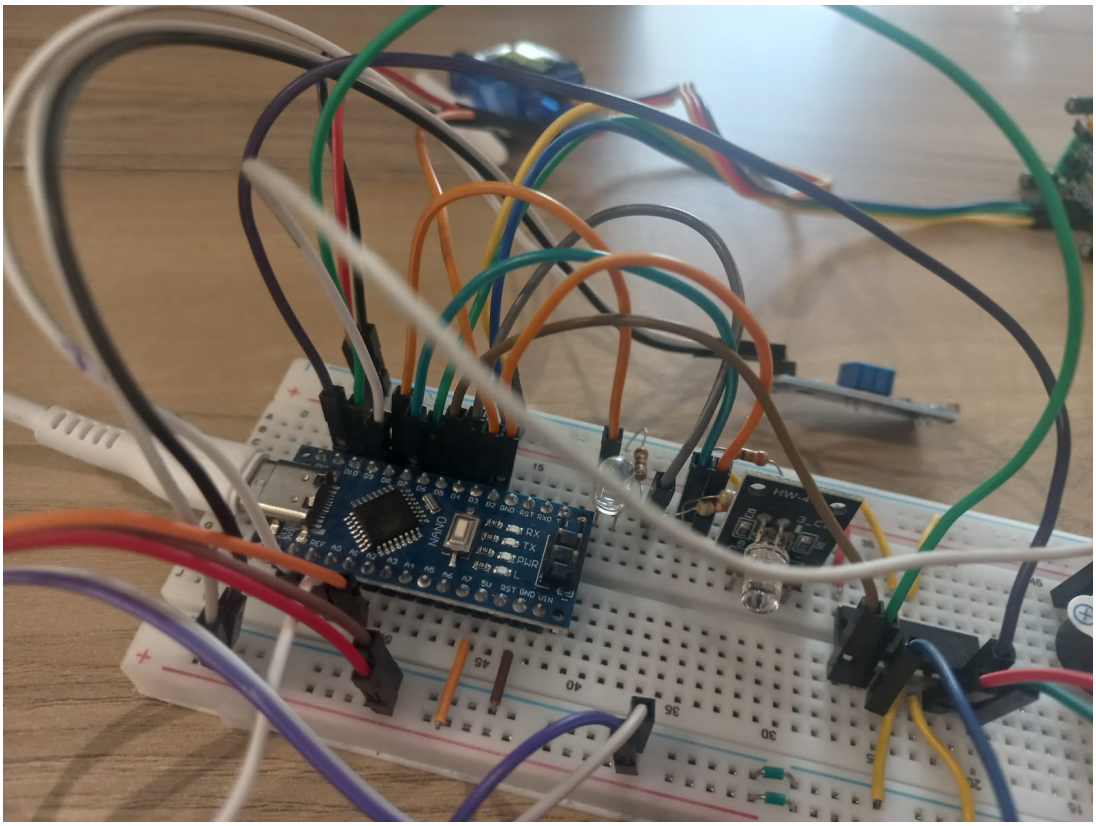
  Once the schematic document was successfully finalized, it was transitioned into the Altium Designer PCB layout environment, where strict component placement strategies were applied to ensure system stability. High-current connections for the DC and servo motors were deliberately isolated from sensitive logic components to prevent electrical noise and power surges from disrupting the microcontroller. Furthermore, the PIR sensor's amplification circuitry was carefully positioned away from the center of the board to protect its highly sensitive resistor from potential interference, ensuring reliable signal integrity across the entire system.

  Additionally, the transition from schematic capture to the physical PCB layout presented unforeseen technical challenges. Syncing the comprehensive schematic data into the PCB environment resulted in significant software instability, including severe system glitches and GPU crashes due to the heavy rendering requirements of the design software. Overcoming these hardware and software toolchain constraints required careful troubleshooting to successfully migrate the design data, ultimately allowing the critical routing phase to proceed without compromising the board's integrity.




GAS1
NTC1
LDR1
J5
J2
J3
SW1
U6
C7
C6
U3
J7
X1
U4
C3
C14
C4
C16
D1
R13
C17
D3
R19
R20
R18
D4
J1
J6
PIR1
C10
C8
R4
R5
R12
BZ1
C5
R3
C1
R1
R6
C17
R16
Smart Nursery Guardian 12/9/2026

PCB


ONAN
RX
TX
PWR